

ENGINEERING · DETAILED DESIGN

Inventre

Low-Level Design

Module-by-module design of the Inventre codebase: frontend routes and components, backend API and server layer, data model, authentication, integration adapters, jobs, configuration and the build-to-production procedure.



DOCUMENT	DATE	STATUS	OWNER
LLD v1.7	23 September 2026	Baseline (as-built)	Inventre Engineering

Companion document: **Inventre — High-Level Design (HLD) v1.7**. Finding IDs (F-01 ... F-23) refer to the HLD's rectification register, Section 11.

Contents

01	Introduction	3
02	Codebase structure & module decomposition	4
03	Frontend design	6
04	Backend API design	9
05	Data model	11
06	Authentication, sessions & authorisation	13
07	Key sequence flows	14
08	Integration adapters	16
09	Background jobs & crons	18
10	Configuration & secrets	20
11	Logging, audit trail & error handling	22
12	Build, test & deployment procedure	23
13	Known limitations & design traps	24

Revision history

Version	Date	Author	Change
1.7	2026-09-23	Engineering	Readability revision: Inter typeface (the previous font file rendered with collapsed word spacing in PDF), larger type scale, every diagram redrawn at reading size with fewer elements. No content change.
1.6	2026-09-23	Engineering	Backup pipeline table (§9.2) and variables (§10.2) updated: midnight schedule, post-commit hook + GitHub push, config mirror, recovery kit.
1.5	2026-09-23	Engineering	Backup pipeline (§9.3), Microsoft 365 variables (§10.2), backup trap updated (§13) after F-11.
1.4	2026-09-23	Engineering	Storefront component split (§3.5), smoke test procedure (§12.2) after F-08 / F-15 (commit <code>cf0ca1d</code>).
1.3	2026-09-23	Engineering	Navigation, cron locks and schedules, validation shape, error boundaries, storefront auto-refresh, repo layout, compose v2 — updated after the F-05...F-22 batch (commits <code>c538354</code> , <code>6e9f4f5</code>).
1.2	2026-09-23	Engineering	Dual-key session verification (§6.1), rotation procedure (§10.3), compose v1 trap (§13) after F-04.
1.1	2026-09-23	Engineering	Cron auth (§4.1, §9), build provenance (§12.4), configuration (§10) and traps (§13) updated after F-01 / F-03 / F-17 / F-21 were deployed (commits <code>934aab7</code> , <code>bff65e6</code>).
1.0	2026-09-23	Engineering	Initial as-built baseline from <code>/root/Inventre</code> (branch <code>feat/ground-stock-availability</code>) and the live production host.

01 Introduction

1.1 Purpose and scope

This Low-Level Design (LLD) explains how each part of Inventre is built so that an engineer can change it safely. It covers the repository at `/root/Inventre`: the Next.js application (frontend and backend in one codebase), its database schema and migrations, its integration adapters, and the operational scripts that run it. It does not cover the Audit ERP's internals.

1.2 Relationship to the HLD

The HLD fixes the boundaries: one modular monolith, Postgres as system of record, adapters for every external system, cron-driven jobs. This LLD stays inside those boundaries and names the files, functions, tables, cookies, environment variables and commands that implement them.

1.3 Conventions

- Paths are repository-relative. `server/` is the server-only domain layer; `lib/` holds code shared by client and server; `app/` is the Next.js App Router tree.
- A "route" is a `route.ts` handler under `app/api`; a "page" is a `page.tsx`.
- Money is integer paise; timestamps are `timestampz`; IDs are UUID v4 unless noted.
- Status pills as in the HLD: Live Partial Gap Planned.

HOW TO USE THIS DOCUMENT

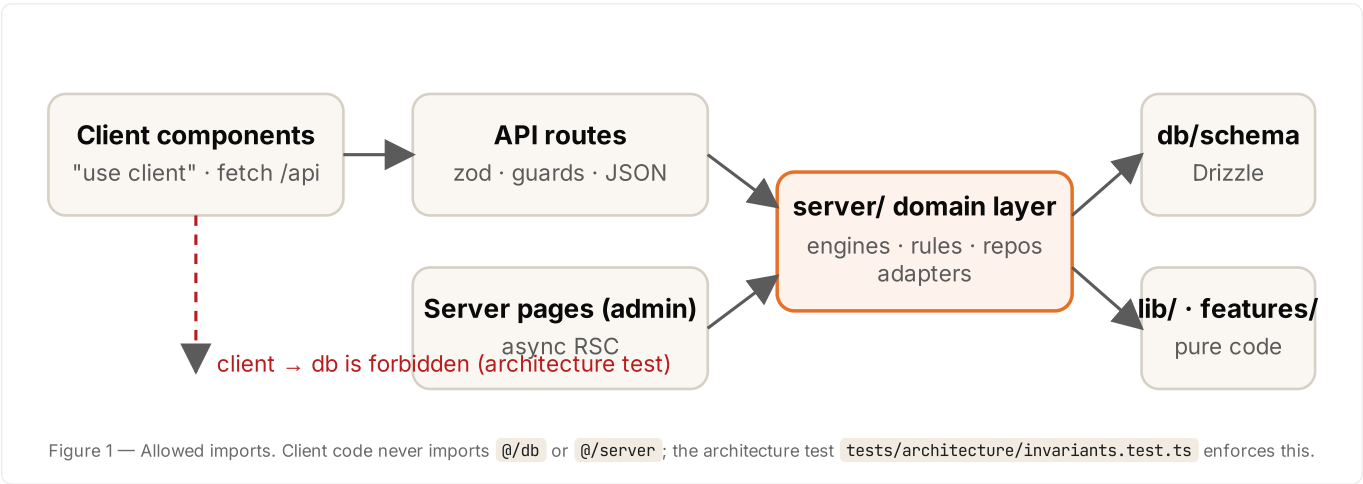
Sections 2 to 6 are reference material (where things are, how they are shaped). Section 7 walks the five flows that matter most. Sections 8 to 12 are operational. Section 13 lists the traps that have bitten in production; read it before touching pricing, grades, migrations or deploys.

02 Codebase structure & module decomposition

2.1 Repository layout

Folder	Size	Contents and rules
app/	152 pages, 233 routes	Next.js App Router. Route groups: <code>admin/(auth)</code> , <code>admin/(protected)</code> . Layouts: <code>root</code> , <code>shop</code> , <code>fees</code> , <code>admin/(protected)</code> , plus one gating layout per admin section.
app/api/	233 <code>route.ts</code>	HTTP API. 150 routes under <code>admin/</code> ; the rest are parent, portal, fees, cron, erp-inbound, support, health, version.
components/	176 files	<code>admin/</code> 88 (own primitive kit in <code>admin/ui</code>) , <code>shop/</code> 38 (<code>pdp/</code> , <code>orders/exchange</code> , <code>orders/missing</code>) , <code>auth/</code> 10, marketing sections at the top level, <code>ui/Modal.tsx</code> .
server/	122 files	Server-only domain layer (<code>import "server-only"</code>). Sub-folders: <code>repos/</code> (data access), <code>erp/</code> (ERPNext client, item feed, media), <code>importers/</code> , <code>jobs/</code> , <code>notify/</code> , <code>mcb/</code> , <code>observability/</code> , <code>admin/</code> . Top-level files are engines and rules (bundle, discount, tax, eligibility, numbering, guards, sessions, ERP bridge, CCAvenue, ground stock).
lib/	~45 files	Isomorphic helpers: <code>cart.tsx</code> (context), <code>jwt.ts</code> , <code>admin-permissions.ts</code> , <code>grade.ts</code> , <code>phone.ts</code> , <code>tax.ts</code> , <code>bookkit-langs.ts</code> , <code>exchange-shared.ts</code> , <code>mcb/mappings.ts</code> , <code>admin-nav.ts</code> (the only navigation definition). Scaffold and empty sub-folders were removed on 2026-09-23 (F-14).
features/	2 domains	<code>bookkit/domain</code> , <code>stock/domain</code> : pure domain logic with tests beside it (eligibility, availability, ground-stock match). The intended home for new pure rules.
db/	schema 3,334 lines; 92 SQL files	<code>schema.ts</code> (Drizzle), <code>client.ts</code> (postgres.js via PgBouncer, pool 30), <code>migrate.ts</code> (custom runner) and its esbuild bundle <code>migrate.js</code> , <code>migrations/*.sql</code> , <code>seed.ts</code> , <code>seed-real-schools.ts</code> .
<code>middleware.ts</code> , <code>instrumentation.ts</code>	—	Edge auth/CSRF gate; unhandled-error telemetry hook.
scripts/	191 files	Operational tooling: deploy, backup, monitor, MCB imports, one-off heals and backfills, E2E driver. Run with <code>tsx</code> ; DB-touching scripts must define <code>main()</code> (no top-level await).
deploy/	—	nginx template, cron.d template, monitoring stack, replica guides, migration notes.
<code>tests/</code> , <code>vitest.config.ts</code>	10 test files, 84 tests	Unit, domain, architecture invariants (section gates, admin route guards, cron auth, navigation slugs, migration prefixes, headers, secrets, client/server boundary, root hygiene), row-count regression.
docs/	—	Architecture plans, flows, runbooks, security (SAST), history, design system.

2.2 Dependency direction



2.3 Module catalogue (server layer)

Module	Purpose
<code>session.ts</code> , <code>parent-guard.ts</code> , <code>admin-guard.ts</code> , <code>fees-auth.ts</code> , <code>support-view.ts</code> , <code>portal-token.ts</code> , <code>cron-auth.ts</code>	Session read/write and the five guard families (parent, admin permission, fees viewer, support view, cron bearer token + Redis job lock).

Module	Purpose
api-handler.ts , parse-body.ts , rate-limit.ts	Request parsing and validation helpers; Redis fixed-window rate limiter.
bundle-engine.ts , bundle-fallback.ts , variant-resolver.ts (repos), variant-code.ts , variant-context.ts	Expand a bundle + selector choices into variant lines; resolve size/colour to a variant; parse ERP variant codes.
discount-engine.ts , cart-coupon.ts , delivery-fee.ts , payment-charges.ts , repos/pricing.ts , lib/tax.ts	Price resolution (item_prices → product_school override → base), discounts, coupons, delivery fee rules by school/grade, gateway charges, GST by pincode.
grade-filter.ts , order-eligibility.ts , return-eligibility.ts , return-line-eligibility.ts , return-qty-cap.ts , exchange-gate.ts , ground-stock-gate.ts , site-access.ts	Business rules: what a student may see and buy, whether an order line may be exchanged or claimed missing, stock gate, site closure.
numbering.ts , invoice-numbering.ts , customer-numbering.ts	Atomic counters per (prefix, period) via UPSERT on numbering_counters ; order numbers SAL-ORD-YYYY-seq continue the ERP sequence.
ccavenue.ts , ccavenue-finalize.ts , ccavenue-settlement-reconcile.ts , repos/payment-settle.ts	Payment gateway crypto, idempotent finaliser, settlement file reconciliation.
erp-config.ts , erp-bridge.ts , erp-drain.ts , erp-poll.ts , erp-reconcile.ts , erp-stranded.ts , erp-jwt.ts , erp-webhook-dispatch.ts , audit-inbound.ts , audit-item-match.ts , erp-customer-orders.ts , erp-import-orders.ts , erp-order-header-refresh.ts , erp-inbound-guard.ts , erp-backfill.ts	Audit ERP bridge, both directions (Section 8.1).
erp/* , importers/* , jobs/*	ERPNext client, item feed, media re-host, education sync; importer per DocType.
ground-stock-sync.ts	Audit Ground Stock → bins (Redis lock, run log).
notify/* , order-confirmation.ts	SMS, e-mail, notification rules, outbound webhooks, idempotent order confirmation.
exchange.ts , missing.ts	Create RTN / MIS requests, dedupe, emit to ERP.
einvoice.ts , ewaybill.ts , gstr1.ts , gstr3b.ts , auto-po.ts	GST documents and reports; low-stock purchase order drafts.
activity.ts , observability/* , cache.ts , redis.ts , storage.ts	Admin activity log; 5xx telemetry; Redis cache with pattern invalidation; single ioredis client; S3 storage.
repos/* (25 files)	Data access per aggregate: products, orders, cart, customers, guardians, grades, inventory, invoices, shipments, media, home, admin-stats, suppliers, purchase-orders, gift-cards, loyalty, school-color-map, categories, payment-settle, product-attribute-groups, variant-resolver.

03 Frontend design

3.1 Route map

Area	Routes	Rendering & guard
Marketing (4)	<code>/</code> , <code>/about</code> , <code>/contact</code> , <code>/experience-store</code>	Server components assembled from <code>components/*</code> sections. <code>/</code> redirects a logged-in parent to <code>/shop</code> . Public cache 60 s / s-maxage 600.
Auth (2)	<code>/login</code> , <code>/reset</code>	Client pages; OTP or password; multi-step recovery (search by student → verify old number → e-mail OTP → add mobile).
Storefront (10)	<code>/shop</code> , <code>/shop/[id]</code> , <code>/shop/cart</code> , <code>/shop/checkout</code> , <code>/shop/orders</code> , <code>/shop/orders/[id]</code> , <code>.../exchange/new</code> , <code>.../exchange/[returnId]</code> , <code>.../missing/new</code> , <code>.../missing/[claimId]</code>	<code>app/shop/layout.tsx</code> (server) requires <code>getCurrentParent()</code> , shows <code>AccessClosed</code> when a student is disabled or the site is closed, then wraps in <code>CartProvider</code> . Pages are client components except the exchange/missing pages (server, <code>force-dynamic</code> , direct DB reads).
Account (2)	<code>/account</code> , <code>/account/change-phone</code>	Client pages against <code>/api/auth/me</code> , <code>/api/auth/change-phone/*</code> .
Admin (125)	<code>/admin</code> (redirect to first accessible page), <code>/admin/login</code> , <code>/admin/(protected)/**</code> (~121 pages in 39 sections), <code>/admin/invoices/[id]/print</code>	Layout: <code>getCurrentUser()</code> must be admin, fee-only accounts → <code>/admin/fee-desk-exit</code> ; <code>AdminShell</code> + <code>VersionWatcher</code> <code>careful</code> . Each section layout wraps pages in <code>SectionGate</code> (<code>canSeePage</code>). 104/122 pages read the DB directly on the server.
Fees (4)	<code>/fees</code> , <code>/fees/login</code> , <code>/fees/users</code> , <code>/fees/receipt/[receiptNo]</code>	Own layout (Archivo + IBM Plex Mono, <code>console.css</code> scoped under <code>.fx</code>), <code>inv_fees</code> session, <code>robots: noindex</code> .
Portal (4)	<code>/portal</code> , <code>/portal/concern</code> , <code>/portal/history</code> , <code>/portal/qr</code> , route <code>/portal/enter</code>	Public, no login; lookup by enrolment or mobile; <code>/portal/qr</code> is the only <code>force-static</code> page.
Support view (1)	<code>/support-view/ended</code> , routes <code>/support-view/exit</code> , <code>/support-view-handoff</code>	Handoff verifies the minted JWT, sets <code>inv_support_view</code> , redirects to <code>/shop</code> ; banner with countdown; exit clears cookie and revokes the mint.

3.2 Layout and rendering rules

- **Root layout** loads Plus Jakarta Sans (`--font-jakarta`), sets `bg-cream text-ink-900`, mounts `SupportViewBanner` (server half reads DB, client half counts down) and `AuthGuard`, which logs the parent out when they leave `/shop/*`.
- **Dynamic by default.** ~115 pages export `dynamic = "force-dynamic"`. No ISR, `unstable_cache` or `revalidateTag`. Headers: `/shop/*` `private no-store; /_next/static` `immutable`.
- **Server actions** exist in four files only (`catalog/build`, `delivery-fee-rules`, `mcb`, `discounts`), each followed by `revalidatePath`. Every other write goes through `/api`. Server actions break open tabs on deploy, which is why `VersionWatcher` exists.
- **Streaming** is limited to the admin `loading.tsx` skeleton and three `Suspense` wrappers around `useSearchParams`. Error boundaries: `app/error.tsx`, `app/not-found.tsx`, `app/shop/error.tsx` (brand styling, retry + back to store; F-15).

3.3 Data fetching on the client

Helper	File	Behaviour
<code>fetchOrNetworkError</code> , <code>errorMessageFor</code>	<code>lib/client-fetch.ts</code>	Timeout, optional retry with backoff, converts network failures to a parent-friendly <code>NetworkError</code> . Used by <code>ExchangeForm</code> and <code>MissingForm</code> (added after the "Failed to fetch" incident).
<code>useFocusRefetch</code>	<code>lib/use-focus-refetch.ts</code>	Refetch on focus / <code>visibilitychange</code> , throttled to 1 s. Used by shop, PDP, cart, checkout, orders, order detail.
<code>auth.me</code> / <code>auth.logout</code> / <code>clearMeCache</code>	<code>lib/auth.ts</code> (client only)	Thin wrapper over <code>/api/auth/*</code> ; session itself is the <code>httpOnly</code> cookie.
<code>VersionWatcher</code>	<code>components/VersionWatcher.tsx</code>	Polls <code>/api/version</code> every 60 s and on focus; reloads on <code>BUILD_ID</code> change unless an input is focused or (careful mode) the page had any edit, then shows a Refresh prompt. Mounted on admin and fees, and since 2026-09-23 on the shop layout in careful mode (F-16).

3.4 Client state

State	Where	Shape
Cart	<code>lib/cart.tsx</code> → <code>CartProvider</code> , <code>useCart()</code>	<code>lines</code> , <code>byStudent</code> (split per sibling), <code>count</code> , <code>total</code> , <code>revision</code> , <code>add</code> , <code>addByVariantId</code> , <code>setQty</code> , <code>remove</code> , <code>refresh</code> , <code>clear</code> , <code>staleRemoved</code> (lines dropped after a grade change). <code>Line = ServerCartLine</code> with <code>bundleSelections[]</code> and <code>attributes[]</code> . Server-backed: every mutation is a call to <code>/api/cart</code> .
PDP selection	<code>components/shop/pdp/SelectionContext.tsx</code>	Chosen size / attributes / qty shared by <code>BuyBox</code> , <code>StickyMobileBar</code> , <code>SizeGuide</code> .
Drafts	<code>lib/product-draft.ts</code> → <code>useProductDraft</code>	Saves PDP and Magic Box picks; logged-in parents to <code>/api/shop/draft</code> , guests to <code>localStorage</code> <code>inventre:draft:{productId}:{studentId}</code> .
localStorage keys	—	<code>inventre.activeStudentId</code> , <code>inv:lastStudentId</code> , <code>inv:orderStatusSeen</code> (drives the "order updated" banner). Never authoritative.
Forms	—	Plain controlled state; zod on the client only in <code>/shop/checkout</code> ; no form or toast library (ad-hoc toasts: <code>QtyCapToast</code> , inline in <code>PDP</code> , <code>ExchangeForm</code> , <code>MissingForm</code>).

3.5 Storefront components that carry business logic

Component	Design
<code>app/shop/[id]/page.tsx</code> (1,150 lines)	Product page: fetch + DTO mapping, gallery, buy box, bundle tree, Magic Box configurator, language chooser wiring, related products; drafts via <code>useProductDraft</code> .
<code>MagicBoxConfigurator.tsx</code> (1,116 lines)	Loads each component via <code>/api/shop/bundle-item</code> ; one variant per component (size, or Mandate × Core × Elective for bookkits via <code>MultiAttributePicker</code>); per-item <code>SizeGuideTable</code> ; restores picks from draft or the existing cart line; detects the colour axis (<code>colorAxisOf</code> , <code>canonicalColor</code>) and soft-confirms mixed house colours; POSTs <code>/api/cart</code> with <code>bundleSelections</code> . The server answers 409 when a house-colour lock is violated.
<code>BundleTree.tsx</code> , <code>CompleteTheKit.tsx</code>	Recursive accordion of bundle contents; cross-sell rail.
<code>lib/bookkit-langs.ts</code> + <code>components/shop/pdp/LangPickers.tsx</code> (<code>KitLangPicker</code> , <code>TemplateLangPicker</code> , <code>LangCard</code> , <code>LangSummary</code>)	Parses ERP names such as "...Bookkit Hindi 2nd Lan Tel 3rd Lan" into 2nd/3rd language pairs and drives the language chooser. Endpoints: <code>/api/shop/bundle-for-variant</code> , <code>/api/shop/kit-mandates/[id]</code> , <code>/api/shop/variant-contents</code> .
<code>components/shop/pdp/*</code>	<code>BuyBox</code> , <code>MultiAttributePicker</code> , <code>SizeGuide</code> , <code>SizeGuideTable</code> , <code>QtyStepper</code> , <code>StickyMobileBar</code> . Size → variant resolution happens server-side (<code>/api/shop/variant</code>).
<code>app/shop/orders/[id]/page.tsx</code> (718 lines) + <code>components/shop/orders/order-detail-types.ts</code> , <code>ItemStatusPill.tsx</code> , <code>CategoryTrackingCard.tsx</code>	Page: data load, status stepper, <code>ShipmentHistory</code> timeline, <code>CCAvenue</code> status polling, exchange and missing banners. Extracted: <code>OrderDetail</code> type and the 7 stages; <code>ItemStatusPill</code> (<code>delivered</code> <code>out for delivery</code> <code>in transit</code> <code>returned</code> <code>pending</code>); <code>CategoryTrackingCard</code> with <code>StageBar</code> , <code>ItemsToggle</code> , shipment matching helpers.
<code>components/shop/orders/exchange/*</code> , <code>missing/*</code>	<code>ExchangeButton/Form/StatusBanner</code> , <code>MissingButton/Form/StatusBanner</code> , <code>RequestStageBar</code> , <code>RequestWindowClosedNotice</code> , <code>RequestBlockedNotice</code> , <code>DuplicateOfNote</code> , <code>CancelRequestButton</code> , <code>QtyCapToast</code> . Photo capture goes through <code>lib/photo-snapshot.ts</code> to avoid stale Android file handles.
<code>lib/school-catalog-gate.ts</code> , <code>server/grade-filter.ts</code>	Catalogue and grade scoping are enforced only on the server (products and cart routes); the client renders the message it is given.

3.6 Admin frontend

- **Shell:** `components/admin/AdminShell.tsx` renders `ADMIN_NAV_GROUPS` from `lib/admin-nav.ts`, the single definition (items carry a page slug; a test checks each slug exists in `ADMIN_PAGES`) (Overview, Sales, Network, People, Catalog, Pricing & Tax, Payment Charges, Buying, Accounting, Engagement, Content, Tools, Settings), filters items by `canSeePage`, mobile drawer, longest-match active item, logout via `/api/admin/auth/logout`. `UniversalSearch` reads the same list and filters by the admin's permissions (F-05 closed).
- **Primitives:** `components/admin/ui/primitives.tsx` (169 importers): `PageHeader`, `Card`, `Stat`, `Badge` + `statusTone`, `Toolbar`, `FilterChips`, `SearchInput`, `DataTable/Th/Td/Tr`, `EmptyState`, `Skeleton`, `Money`; `Button`, `IconBtn` in `primitives-client.tsx`.
- **Modules:** products (variants, schools, combo prices, export, price list), catalog hub (guided uniform/bookkit builders, bundles, attributes, bulk pricing, stock adjust/ledger/reconcile/transfer, BOMs, Ground Stock), orders (list, detail, builder, export, shipments, invoices, returns, exchanges), people (students browser + import, customers, guardians, schools wizard, grades), MCB, settings (users, roles, ERP bridge/sync, OTP, notifications, payments, shipping, webhooks), activity log with record history, 11 reports, purchasing, discounts, gift cards, content, reviews, testimonials, concerns, OTP logs.
- **Permissions in UI:** read-only roles see disabled write controls via `canWriteAsAdmin`; API enforces with `requirePermission`.

3.7 Fee console

`app/fees/page.tsx` (server) checks `getFeesViewer` and one of `fees.read` | `fees.write` | `mcb.read` | `mcb.write`, reads academic years from `mcb_fee_payments`, and renders `FeeConsole.tsx` (1,555 lines, client). Views (schools, heads, coverage, ageing, students, receipts) each call

`/api/admin/mcb/fees?view=...`. Colour roles: billed = blue, in = green, out = orange, concession = plum; no fifth accent. Receipts: stored copy first, live MCB fallback (`server/mcb/receipts.ts`), printable `/fees/receipt/[no]` .

3.8 Design tokens

COLOUR

brand `#E47127` (50–900), ink `#0A0A0A` (50–900), cream `#FAF7F2`. Status: emerald (success), red (error), amber (low stock). Light mode only; dark panels are design, not a theme.

TYPE & MOTION

Plus Jakarta Sans for display and body; fluid `display-xl/lg/md` via `clamp()`. `framer-motion` in 50 files; CSS keyframes in admin; global `prefers-reduced-motion` override.

RESPONSIVE & A11Y

Mobile-first Tailwind (`lg: 457`, `sm: 212` uses); `StickyMobileBar`, admin drawer. Brand focus ring; 105 `aria-label`s; no automated a11y checks (F-15). No PWA manifest.

04 Backend API design

4.1 Handler conventions

- **Shape:** every handler is a Next.js route handler exporting `GET/POST/PATCH/DELETE`. Bodies and queries are validated with zod (142 route files) through `server/api-handler.ts` `parseJson` / `parseQuery` (`parseBody` in `server/parse-body.ts` is an alias). One 400 payload everywhere: `{error: "<messages joined>", details: [{path, message}], issues: ZodIssue[]}` (F-18).
- **Errors:** `{error: string}` with the HTTP status; successes commonly `{ok: true, ...}`. Middleware returns `{error: "Unauthorized"}` 401, `{error: "Forbidden (CSRF)"}` 403, and a read-only 403 in support view. `failJson` (5 parent routes) also records the failure in `storefront_events`.
- **Guards:** first statement of the handler. Parent: `requireParent()`. Admin: `requirePermission("<slug>.read|write")` (141 routes), `requireAnyPermission` (3), `requireAnyWritePermission` (upload), legacy `requireAdmin` (2). Fees: `getFeesViewer()`. Cron: `requireCron(req)` then `acquireCronLock(name, ttl)` from `server/cron-auth.ts` (see 9.2). Service: `SUPPORT_VIEW_SERVICE_TOKEN`, `SYNC_STAGING_KEY`.
- **Side effects after writes:** `logActivity` (14 admin routes), cache invalidation (`invalidateCatalog`), `enqueueOrderEvent`, notifications through `server/notify/event-bus.ts`.
- **Relative redirects:** route handlers behind nginx must return a relative `Location` because `req.url` is `0.0.0.0:3000`.

4.2 Endpoint groups

Group (prefix)	Auth	Endpoints and behaviour
<code>/api/auth/*</code>	Public + rate limit	<code>otp/request</code> , <code>otp/verify</code> , <code>login</code> (password), <code>logout</code> , <code>me</code> (GET/PATCH), <code>register</code> , <code>first-time/*</code> , <code>forgot-password(/complete)</code> , <code>reset-password</code> , <code>change-phone/{request,verify}</code> , <code>accept-tc</code> , <code>phone-status</code> , <code>recover/*</code> (search, verify-old, e-mail OTP, add-mobile, confirm). OTP state in Redis keys <code>otp:<phone></code> , <code>otp:attempts:<phone></code> ; every send logged in <code>otp_logs</code> .
<code>/api/shop/*</code>	Parent (some public)	<code>products</code> , <code>products/{slug}</code> , <code>categories</code> , <code>bundles/{productId}</code> , <code>bundle-item</code> , <code>bundle-for-variant</code> , <code>kit-mandates/{productId}</code> , <code>variant</code> , <code>variant-contents</code> , <code>draft</code> (GET/POST/DELETE), <code>orders/{id}/exchange/{returnId}/cancel</code> , <code>orders/{id}/missing/{claimId}/cancel</code> . Reads go through the Redis cache; school and grade scoping applied server-side.
<code>/api/cart*</code>	Parent	<code>cart</code> GET/POST/PATCH/DELETE (add resolves bundles through <code>expandBundle</code> ; qty capped to one for free bookkits and Magic Boxes; house-colour lock → 409), <code>cart/apply-coupon</code> , <code>cart/shipping</code> (delivery fee rules).
<code>/api/checkout/ccavenue/*</code>	Parent; callback signature-only	<code>create-order</code> (creates order + payment rows, returns encrypted form), <code>callback</code> (AES-decrypt, finalise, 303 to order page or JSON for server notify), <code>status/{orderId}</code> (Status API poll from the order page).
<code>/api/orders*</code> , <code>/api/returns*</code> , <code>/api/missing</code> , <code>/api/addresses</code> , <code>/api/reviews</code>	Parent	Orders list/detail scoped to the family graph; <code>orders/{id}/shipments/{sid}/refresh</code> ; returns (exchanges, <code>returns.kind</code>) and missing claims create requests, dedupe per sales order, emit to ERP; <code>returns/upload</code> for photos (60 MB, outside the middleware matcher).
<code>/api/portal/*</code>	Public / parent / admin	<code>search</code> , <code>concerns</code> (GET/POST, emits <code>concern.created</code>), <code>upload</code> public; <code>summary</code> parent; <code>token</code> admin-only (HMAC portal tokens, 120-day TTL).
<code>/api/admin/**</code> (150)	Admin permission	Catalogue (products, images, variants-bulk, grades, schools, combo-prices, bulk, export, price-list), attributes, bundles, BOMs, categories, item-prices, price-lists, discount-rules, website-cart-coupons; orders (detail, replace, refresh-cca, bulk-refresh-cca, import-from-erp, recompute-percentages), payments, shipments (+ e-way bill), invoices (+ e-invoice), returns, exchanges; people (<code>data/students guardians schools grades otp_logs</code> , students import/export/bulk flags, customers); stock (adjust, transfer, reconcile, auto-po, ground-sync), warehouses, suppliers, purchase-orders/receipts/invoices; reports (sales, items, customers, fulfillment, stock, gst, gstr1, gstr3b, hsn); settings (otp, payment-charges, site-access, site-closure), users (+ permissions), roles, webhooks, notification-rules, notifications resend, erp (pull, sync-items, sync-education, rehost-images), erp-bridge (replay, send-now), mcb (data, export, fees, receipts), fees/users, content, faqs, testimonials, reviews, concerns, gift-cards, loyalty, upload, export/[type], sync-staging.
<code>/api/fees/auth/*</code>	Public + rate limit	Fee-desk login/logout → <code>inv_fees</code> .
<code>/api/erp/webhooks(/exchange /missing)</code>	HMAC X-ERP-Signature	Inbound from Audit ERP. Root receiver inserts into <code>erp_webhook_events</code> (idempotent on <code>event_id</code>) and returns 200; processing is deferred to <code>erp-webhook-drain</code> .
<code>/api/cron/*</code> (12)	Shared secret	See Section 9.
<code>/api/support/mint-view</code>	Service bearer	Mints a read-only view-as-parent JWT; logged in <code>activity_log</code> .
<code>/api/health</code> , <code>/api/version</code>	Public	DB + Redis ping → 200/503; <code>{buildId}</code> from <code>.next/BUILD_ID</code> .

Group (prefix)	Auth	Endpoints and behaviour
/api/contact , /api/newsletter , /api/gift-cards/lookup , /api/loyalty/balance , /api/payment-charges	Public / parent	Marketing forms (rate-limited), gift card lookup, loyalty balance, gateway charge table.

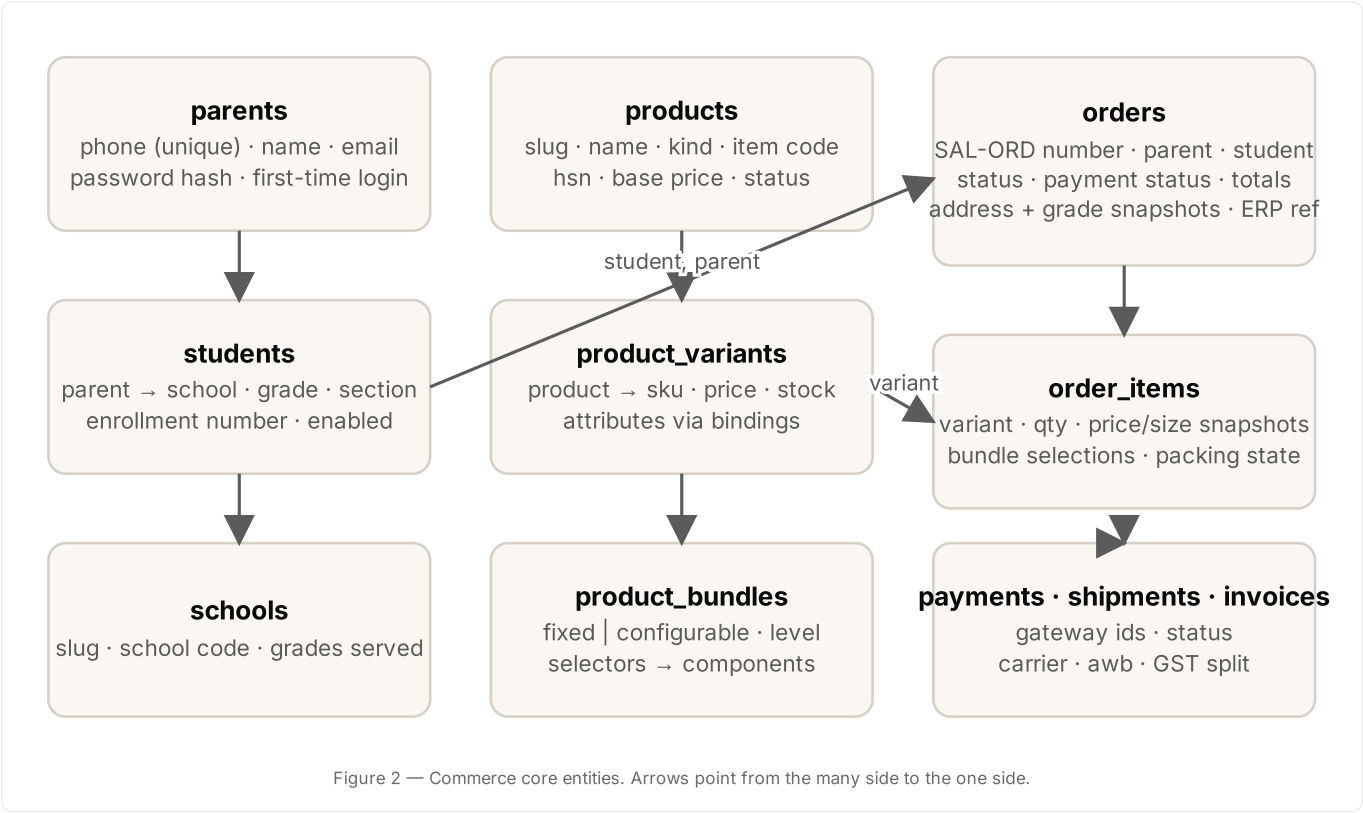
4.3 Representative handler

```
// app/api/cart/route.ts (shape, abbreviated)
export async function POST(req: Request) {
  const parent = await requireParent();           // 401 if no inv_session
  const body = await parseJson(req, AddSchema);   // zod → 400 {error, details}
  if (body instanceof NextResponse) return body;
  const student = await assertStudentOfParent(parent.id, body.studentId);
  await assertGradeEligible(student, body.variantId); // server/grade-filter.ts
  const lines = body.bundleId
    ? await expandBundle({ bundleId: body.bundleId, choices: body.bundleSelections })
      : [{ variantId: body.variantId, qty: capToOne(body.qty) }];
  await assertHouseColourLock(student, lines);      // 409 on mismatch
  const cart = await cartRepo.addLines(parent.id, student.id, lines);
  return NextResponse.json({ ok: true, cart });
}
```

05 Data model

96 Drizzle tables and 34 enums in `db/schema.ts`, plus 12 tables created by raw SQL migrations. The entity diagram below shows the commerce core; the tables that follow list key columns per domain.

5.1 Commerce core



5.2 Domain tables and key columns

Domain	Tables (key columns in brackets)
Identity & RBAC	<code>users</code> (email, password_hash, role super ops school_admin, role_id, school_id) · <code>admin_roles</code> · <code>admin_role_permissions</code> (SQL 0044-0047) · <code>admin_user_permissions</code> (grant / revoke per key) · <code>otp_logs</code> (phone, purpose, gateway response, status)
Family graph	<code>parents</code> · <code>guardians</code> · <code>student_guardian_links</code> · <code>student_siblings</code> · <code>student_addresses</code> · <code>addresses</code> (type enum). A phone may map to many guardians and students; queries always walk the graph, never assume one parent per student.
Schools & grades	<code>schools</code> · <code>grades</code> · <code>school_grade_mappings</code> (ERP label ↔ real grade) · <code>school_uniform_mappings</code> · <code>school_coordinators</code> · <code>school_color_map</code> (house colours) · <code>companies</code> · <code>customer_groups</code>
Catalogue	<code>categories</code> · <code>products</code> (kind enum: magic_box, kit, sub_bundle, uniform, accessory, book, consumable, excluded) · <code>product_variants</code> · <code>product_images</code> · <code>product_badges</code> · <code>product_grades</code> · <code>product_school</code> (per-school visibility + price override) · <code>product_attributes</code> / <code>_values</code> / <code>_bindings</code> / <code>product_variant_attributes</code> · <code>product_drafts</code> · <code>product_uoms</code> · <code>product_barcodes</code> (SQL 0011)
Bundles	<code>product_bundles</code> (type fixed configurable, level, pricing_mode) · <code>bundle_selectors</code> (one_of multi) · <code>bundle_components</code> · <code>bundle_configs</code>
Pricing & tax	<code>price_lists</code> (applies_to enum) · <code>item_prices</code> (valid_from/until) · <code>coupons</code> · <code>discount_rules</code> · <code>discount_usages</code> · <code>website_cart_coupons</code> / <code>_usages</code> · <code>tax_rates</code> · <code>hsn_codes</code> · <code>delivery_fee_rules</code> / <code>_grades</code>
Stock	<code>warehouses</code> · <code>bins</code> (actual, reserved, min) · <code>stock_ledger</code> (reason enum, append-only) · <code>ground_stock_sync</code> / <code>_runs</code>
Commerce	<code>carts</code> · <code>cart_items</code> · <code>orders</code> · <code>order_items</code> · <code>payments</code> · <code>payment_schedules</code> · <code>payment_entries</code> · <code>modes_of_payment</code> · <code>shipments</code> / <code>_items</code> · <code>invoices</code> / <code>_items</code> · <code>numbering_counters</code> (prefix, period, value) · <code>order_notifications</code> · <code>communications</code>
After-sales	<code>returns</code> (kind, source, status enum requested → approved → received → refunded rejected, requested_component_path, replacement_mode, duplicate_of, photos) · <code>return_items</code> · <code>missing_item_claims</code> / <code>_items</code> · <code>concerns</code> (concern_number CON-, category, team, audit_ref) · <code>concern_messages</code> · <code>contact_submissions</code>

Domain	Tables (key columns in brackets)
Integration	<code>erp_outbound_queue</code> (status pending sending sent failed cancelled) · <code>erp_outbound_dlq</code> (SQL 0030) · <code>erp_webhook_events</code> (SQL 0031/0034) · <code>webhook_endpoints</code> · <code>webhook_deliveries</code> · <code>background_jobs</code> · <code>schema_erp.*</code> (sync_state, students created by migrations; nine mirror tables owned by the ERP) · <code>mcb_students</code> , <code>mcb_fee_payments</code> (SQL 0021) · <code>mcb_fee_transactions</code> , <code>mcb_fee_txn_sync</code> (SQL 0074)
Buying	<code>suppliers</code> · <code>purchase_orders</code> / <code>_items</code> · <code>purchase_receipts</code> / <code>_items</code> · <code>purchase_invoices</code> / <code>_items</code>
Engagement & CMS	<code>reviews</code> · <code>wishlists</code> · <code>testimonials</code> · <code>faqs</code> · <code>content_blocks</code> · <code>media</code> · <code>loyalty_ledger</code> · <code>gift_cards</code> / <code>_redemptions</code> · <code>notification_rules</code>
Ops	<code>activity_log</code> (actor, action, entity, before/after) · <code>audit_log</code> · <code>system_settings</code> (e.g. <code>site.access_closed</code>) · <code>storefront_events</code> (SQL 0072; kind api.5xx etc.)

5.3 Lifecycle enums

Enum	Values
<code>order_status</code>	placed → confirmed → packed → shipped → delivered · returned · cancelled (plus <code>display_status</code> text derived from ERP shipments and RTO)
<code>payment_status</code>	pending · paid · failed · refunded
<code>shipment_status</code>	draft · packed · shipped · out_for_delivery · delivered · returned · cancelled
<code>return_status</code>	requested · approved · received · refunded · rejected (storefront stepper: RAISED → ... → DELIVERED synced from the ERP)
<code>invoice_status</code>	draft · submitted · paid · partially_paid · overdue · cancelled
<code>account_status</code>	active · blocked · pending (no "disabled")
others	<code>user_role</code> , <code>school_status</code> , <code>product_status</code> , <code>review_status</code> , <code>product_badge</code> , <code>attribute_type</code> , <code>price_list_applies_to</code> , <code>stock_ledger_reason</code> , <code>gst_treatment</code> , <code>address_type</code> , <code>einvoice_status</code> , <code>discount_type</code> , <code>discount_applies_to</code> , <code>bundle_type</code> , <code>bundle_level</code> , <code>product_kind</code> , <code>bundle_pricing_mode</code> , <code>selector_type</code> , <code>return_condition</code> , <code>refund_method</code> , <code>background_job_status</code> , <code>supplier_status</code> , <code>purchase_order_status</code> , <code>purchase_invoice_status</code> , <code>payment_method</code> , <code>payment_direction</code> , <code>communication_kind</code>

5.4 Grade handling

TRAP

`students.grade` is canonical and is sent to the ERP verbatim. The legacy ERP labels carry a +3 offset ("Grade 4" means Grade 1); `lib/grade-translate.ts` and `school_grade_mappings` translate at the boundary, and bulk-imported rows store the real grade. Never read `orders.grade_snapshot` as the truth for a student's grade.

5.5 Migrations

- Runner:** `db/migrate.ts` (bundled to `db/migrate.js`) is a custom file-based migrator, not drizzle-kit. It connects on `DATABASE_DIRECT_URL` (falls back to `DATABASE_URL`), records applied filenames in `__schema_migrations(filename, applied_at)`, marks 0000–0005 applied by fingerprint if their tables already exist, and applies each pending file in its own transaction. `MIGRATE_SKIP_HISTORICAL_SEEDS=1` skips `0037_catalog_data_backfill.sql` (CI).
- Boot:** `scripts/start.sh` runs the migrator when `RUN_MIGRATIONS_ON_START=1` (default); a failure aborts the container unless `MIGRATIONS_REQUIRED=0`; then `exec node server.js`.
- Rules:** always write a migration (never `db:push` against a shared DB); enum additions and CHECK changes in one transaction; guard name-based constraints because production has `erp_*_pkey` names that differ from dev (the 0014 incident 502'd the site).
- Hygiene:** Gap ten duplicated numeric prefixes, a stray nested `db/migrations/migrations/` copy, journal listing only 0000–0001, and ERP-owned `erp.*` mirror tables that a rebuilt database lacks (F-19).

06 Authentication, sessions & authorisation

6.1 Tokens and cookies

Cookie	Kind	Payload	Issued by	Attributes
inv_session	parent	{sub, kind, schoolId?, role?, phone?, iat, exp}, HS256 (jose), 7-day TTL. Signed with JWT_SECRET; verified against JWT_SECRET then	createParentSession after OTP verify / password login	httpOnly · SameSite=Lax · Secure i production (disable only with COOKIE_INSECURE=1 and
inv_admin	admin	JWT_SECRET_PREVIOUS (lib/jwt.ts getVerifyKeys, mirrored in middleware.ts), so the secret rotates without logging anyone out	createAdminSession after bcrypt compare + rate limit	ALLOW_INSECURE_COOKIES_IN_PROD · path /
inv_fees	fees		/api/fees/auth/login	
inv_support_view	support	Separate JWT with SUPPORT_VIEW_JWT_SECRET, short TTL, references the mint	Handoff route after POST /api/support/mint-view	Same attributes; enables read-only mode only when SUPPORT_VIEW_ENABLED=true

TRAP

signSession keys off JWT_SECRET only. A token signed with AUTH_SECRET verifies nowhere and fails silently. The fee console outage of 2026-08-27 was exactly this.

6.2 Reading the session

- server/session.ts: getSession(), getCurrentUser() (admin cookie first, then parent; React cache d per request), getCurrentParent() (parent cookie only, so a stale admin cookie never leaks into the storefront), destroy*Session().
- Admin permissions = (admin_roles → admin_role_permissions) U user grants – user revokes (admin_user_permissions.granted). Keys are <slug>.read / <slug>.write for the ~34 slugs in lib/admin-permissions.ts ADMIN_PAGES; school_admin is read-only; schoolScope / assertSchoolAccess narrow data to the admin's school.
- Fees: getFeesViewer() accepts an admin or a fees session and checks fees.read|write, mcb.read|write. fees-admin may manage fee-desk users but is fenced to MANAGED_ROLES, so no escalation.

6.3 Edge middleware (middleware.ts)

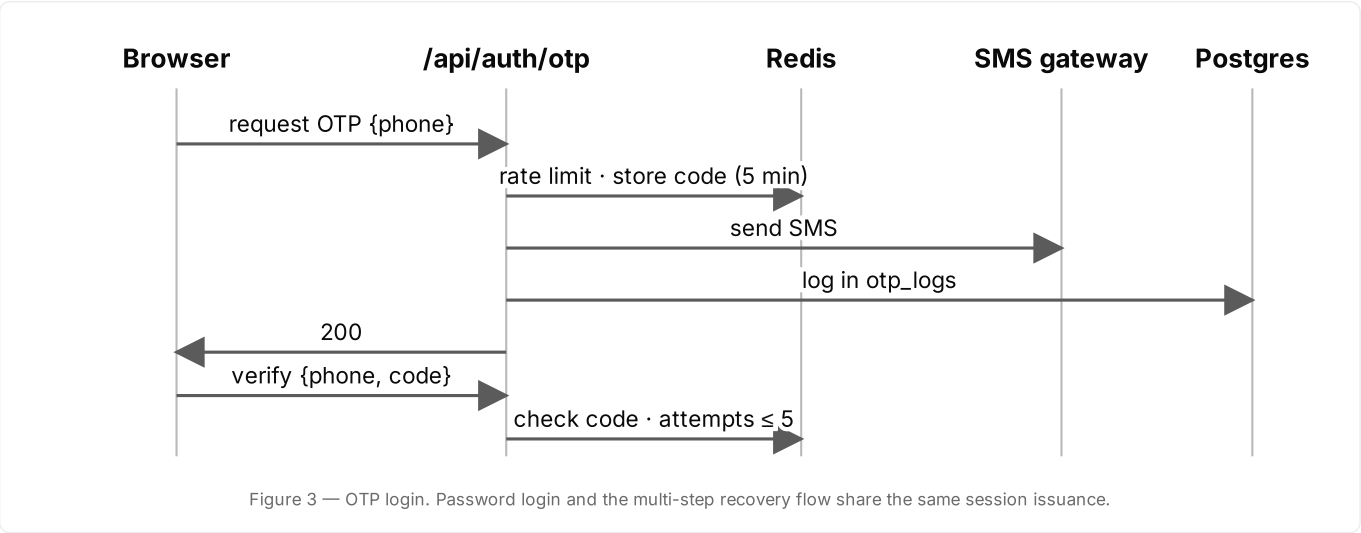
- Strip any client-sent x-inv-support-view header.
- Page gating: /admin/* (except /admin/login) needs a valid inv_admin → else redirect /admin/login?from=...; /shop/* needs inv_session → else /login.
- API gating: /api/admin/* admin token; fee APIs admin or fees; parent APIs (orders, returns, addresses, cart, reviews, checkout) parent token, except the CCAvenue callback. A token of the wrong kind is rejected → 401 JSON.
- CSRF: for POST/PUT/PATCH/DELETE under /api/admin/, /api/fees/, /api/auth/, /api/cart, /api/orders, /api/returns, /api/addresses, /api/reviews, /api/checkout, the Origin or Referer host must equal the request host → else 403. Exempt: /api/webhooks, /api/checkout/ccavenue/callback.
- Support view: valid inv_support_view + enabled flag → every mutating method returns 403.
- Matcher excludes /api/admin/upload and /api/returns/upload (Next's 10 MB middleware body cap); those routes authenticate themselves.

6.4 Rate limiting

server/rate-limit.ts implements a fixed window with Redis INCR + EXPIRE (the comment says sliding; it is fixed). 25 routes use it: OTP request (otp:req:ip:<ip> 30/h), login, recovery, contact, newsletter, fees login, admin login.

07 Key sequence flows

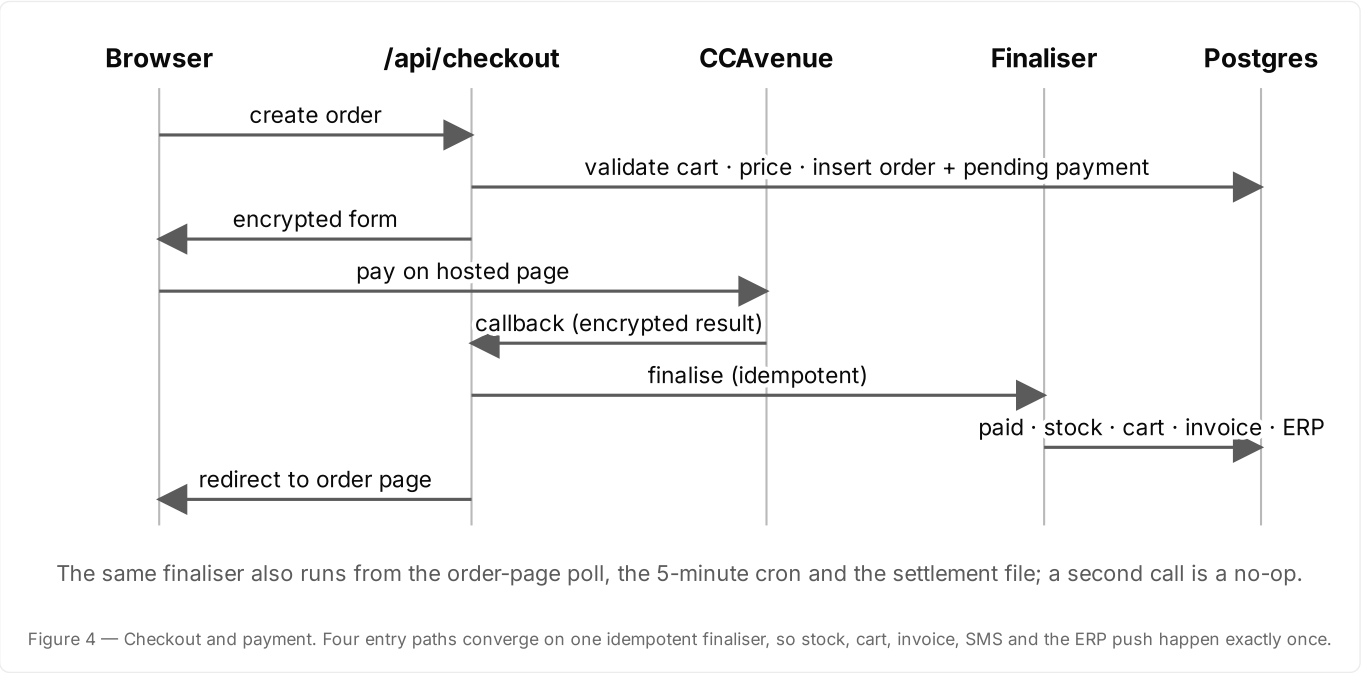
7.1 Parent login by OTP



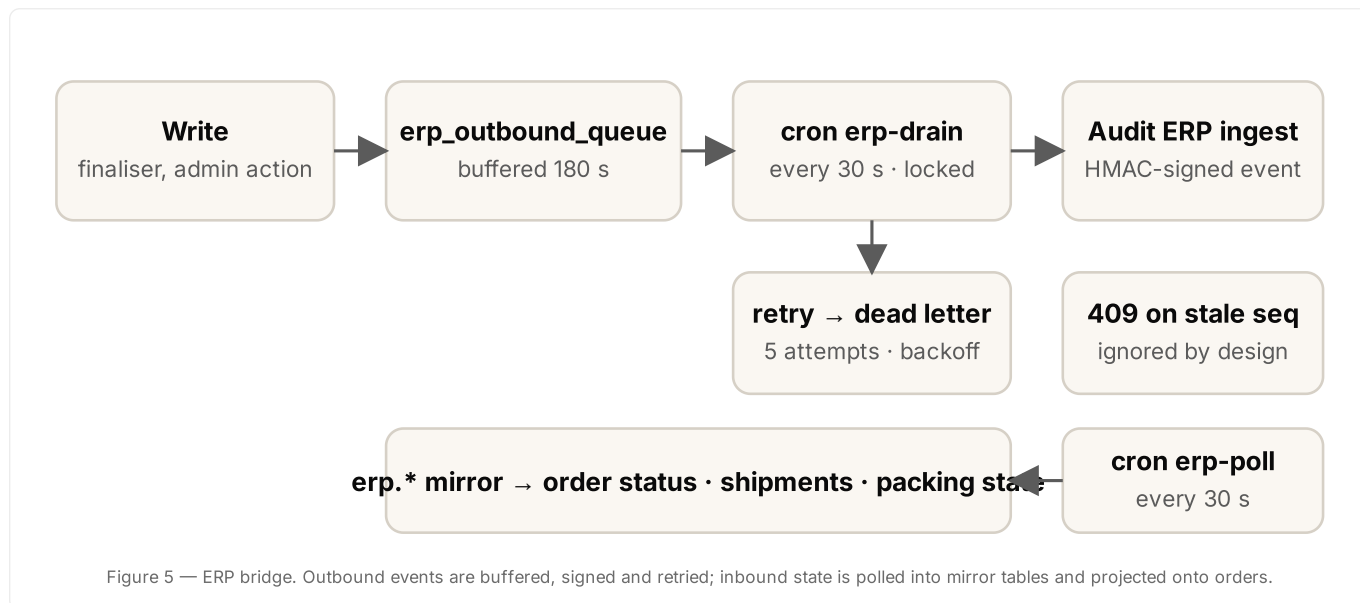
7.2 Catalogue resolution for a student

- 1. GET /api/shop/products?studentId= → requireParent , assert the student belongs to the family graph.
- 2. lib/school-catalog-gate.ts : if the school's catalogue is disabled (e.g. YIPS), return the message and no products.
- 3. server/grade-filter.ts : products visible = product_school rows for the school ∩ product_grades for the student's grade (merchandise such as bags and bottles visible to all schools).
- 4. server/repos/pricing.ts : price per line = item_prices (valid window) → product_school.price_override → product_variants.price / products.base_price .
- 5. server/ground-stock-gate.ts : when the gate is on, variants whose bin is uncounted or negative render sold out.
- 6. Result cached in Redis under products:school:<id>:grade:<g> ; any catalogue write calls invalidateCatalog() .

7.3 Cart → checkout → payment



7.4 Order → Audit ERP and back



Event types: `order.created|updated|cancelled|deleted`, `payment.updated`, `student|guardian|address.upserted`, `exchange.requested`, `missing.requested`, `concern.created`. `order.deleted` is emitted directly (never queued) from the admin DELETE route. Sub-items for Magic Boxes are rebuilt from `bundle_components` when `bundle_selections` is null (all-leaf guard spares bookkits). The audit side drops sub-items for sealed or dispatched orders yet still returns 200, so verify in the audit DB after a heal.

7.5 Exchange / missing request

- Eligibility** (`server/exchange-gate.ts`, `return-eligibility.ts`, `return-line-eligibility.ts`): order paid and delivered; window = 7 days from the day the last item was delivered (open until all arrive); Magic Box exempt at box level (per-component requests only, no whole-box); line `packing_state` must be delivered; pending lines are badged and not selectable; duplicate per sales order → `duplicate_of`.
- Drill-down**: bookkit → category (sub_bundle) → book; whole box / whole category / individual item via `requested_component_path`. Qty capped to ordered qty (`return-qty-cap.ts`).
- Create**: POST `/api/returns` (kind exchange) or `/api/missing`; photos via `/api/returns/upload` to R2; one RTN per submission; number minted by the ERP (RTN-M- / MIS-M-) or locally (RTN-YYYY) per `RETURNS_NUMBER_SINGLE_SOURCE`.
- Emit** `exchange.requested` / `missing.requested` through the queue; status returns via `/api/erp/webhooks/{exchange,missing}` → `erp_webhook_events` → drain → `returns.status`; storefront stepper and banners read it; customer cancel via `/api/shop/orders/{id}/{exchange,missing}/{id}/cancel`.
- Copy rule**: no pickup date is promised; store-pickup schools (KLINK, QLPHP) say "the store team".

08 Integration adapters

8.1 Audit ERP bridge

Aspect	Design
Configuration	<code>server/erp-config.ts</code> is the only reader: <code>ERP_TARGET=staging prod</code> selects <code>STAGING_ERP_*</code> or <code>PROD_ERP_*</code> (<code>INGEST_URL</code> , <code>API_BASE_URL</code> , <code>WEBHOOK_SECRET</code> , <code>POLL_USER</code> , <code>POLL_PASS</code>); tunables buffer 180 s, drain chunk 50, max attempts 5, concurrency 5, stuck-sending 90 s. Note: the staging URL in <code>.env.deploy</code> points at the audit <i>production</i> host.
Outbound	<code>erp-bridge.ts</code> builds the envelope from orders, items, payments, students, guardians, bundles; signs with HMAC-SHA256 (X-Ecom-Signature); 10 s timeout; each attempt logged in <code>webhook_deliveries</code> (endpoint "erp-bridge"). <code>erp-drain.ts</code> claims rows, recovers stuck sending, retries with exponential backoff, dead-letters after 5 attempts. Admin can <code>replay</code> or <code>send-now</code> a row. <code>erp-stranded.ts</code> e-mails <code>ALERT_EMAIL</code> for events older than <code>STRANDED_GRACE_HOURS</code> .
Inbound poll	<code>erp-poll.ts</code> : delta poll with watermarks in <code>erp.sync_state</code> ; JWT from <code>erp-jwt.ts</code> (login then cached); upserts <code>erp.customers students sales_orders ... outward_shipments packing_units</code> ; <code>deriveStatusForErpOrderName</code> recomputes <code>orders.status</code> , <code>display_status</code> , <code>delivered_percent</code> , per-line <code>packing_state</code> ; S-Rocket/RTO overrides are timestamp-aware. Kill switches <code>ERP_INBOUND_ENABLED</code> , <code>ERP_INBOUND_ORDERS_POLL_ENABLED</code> .
Inbound webhooks	<code>/api/erp/webhooks*</code> verify <code>sha256=<hex></code> over the raw body, store, ack 200; <code>erp-webhook-drain</code> processes with <code>FOR UPDATE SKIP LOCKED</code> (<code>erp-webhook-dispatch.ts</code> , <code>audit-inbound.ts</code>). Manual audit-created requests reach Inventre this way.
Import / heal tooling	<code>erp-import-orders.ts</code> (ERP-only orphans), <code>erp-customer-orders.ts</code> (parent order detail from ERP), <code>erp-backfill.ts</code> , <code>scripts/reemit-orders-batch.ts</code> , <code>backfill-erp-reemit.ts</code> , <code>replay-erp-fulfillment-grade.ts</code> .

8.2 CCAvenue

- `server/ccavenue.ts`: AES-128-CBC with MD5-derived key and fixed IV (gateway specification); env `CCAvenue_MERCHANT_ID`, `ACCESS_CODE`, `WORKING_KEY`, `API_BASE`, `REDIRECT_URL`, `CANCEL_URL`, plus `CCAvenue_STATUS_*` for the Status API.
- `server/ccavenue-finalize.ts` `finalizeOrderPayment({source})`: sources `callback`, `status-poll`, `cron-reconcile`, `settlement-reconcile`; results `no-change` (`already_finalized`, `still_pending`, `unknown_status`, `deferred_duplicate_submit`) or a transition. Handles "Payment already done" as success. Not `server-only` so tsx tools can import it.
- `server/ccavenue-settlement-reconcile.ts`: reads .xlsx files from `CCAvenue_SETTLEMENT_DIR` (container mount `/settlements` ← `host /home/ccavenue-sftp/settlements`), matches tracking IDs, marks settled, heals captured-but-stuck orders.
- Admin tools: `orders/[id]/refresh-cca`, `orders/bulk-refresh-cca`, `payments/ccavenue` page; `scripts/remediate-ccavenue-stuck.ts`.

8.3 Notifications

- `server/notify/sms.ts`: Arihant Global HTTP API (`SMS_API_URL/USERNAME/PASSWORD`) for OTP; transactional credentials `SMS_TRANS_API_*`; console fallback without credentials; `DEV_OTP`, `OTP_BYPASS_CODE`, `OTP_TEST_BYPASS` for test; admin toggles in `otp-toggles.ts`. Gap non-OTP sends no-op (F-09).
- `server/notify/email.ts`: SMTP (nodemailer) → Resend → console.
- `server/notify/event-bus.ts`: `notification_rules` fan out to e-mail/SMS; `webhook_endpoints` receive HMAC-signed outbound webhooks retried by `retry-webhooks`; templates in `notifications.ts`, `notification-template.ts`; `order-confirmation.ts` idempotent via `order_notifications`.

8.4 MyClassBoard

- `scripts/import-from-mcb.ts --prune` (01:30): students + receivables for 13 branches into `mcb_students`, `mcb_fee_payments`; window `MCB_FEE_WINDOW_DAYS` (600, because PAID mutates after VoucherDate); school resolved per student by enrolment (one branch spans CBSE/CIE/pre-school).
- `scripts/import-mcb-receipts.ts` (02:30): per-student `GET_StudentFee_Transactions` into `mcb_fee_transactions`, progress in `mcb_fee_txn_sync`.
- `server/mcb/receipts.ts`: stored-first lookup, live fallback; `lib/mcb/mappings.ts` branch/head codes. Env: `MCB_API_BASE`, `MCB_API_KEY`, `MCB_TOKEN_ID`, `MCB_BRANCH_IDS`, `MCB_ORGANISATION_ID`, `MCB_ACADEMIC_YEAR_IDS`.
- Rules: leavers (`Status1`) drop out of *outstanding* only and only for the live year; receipt-only heads shown as "collected only"; ageing buckets by days past DueDate (IST).

8.5 ERPNext, Ground Stock, storage, Redis

- ERPNext**: `server/erp/erpnext-client.ts`, `items-feed.ts`, `puller.ts`, `media-rehoster.ts`; importers per DocType in `server/importers`; jobs in `server/jobs` write a `background_jobs` row and run detached with `setImmediate` (no worker; `enqueue.ts` is a stub).
- Ground Stock**: `server/ground-stock-sync.ts` under a Redis lock; writes bins and `ground_stock_sync_runs`; gate in `ground-stock-gate.ts`; matcher in `features/stock/domain/ground-stock-match.ts` (waist-length sizes, Beige=Brown fold).
- Storage**: `server/storage.ts` (S3 SDK, `forcePathStyle`; `S3_ENDPOINT/REGION/BUCKET/ACCESS_KEY_ID/SECRET_ACCESS_KEY/PUBLIC_URL`); media rows in `media`; R2 public reads.

- **Redis:** `server/redis.ts` single `ioredis` client on `globalThis` behind a lazy proxy with `reconnect/backoff` (post-leak fix 2026-05-26);
`server/cache.ts` `cached(key, ttl, fn)`, `invalidate`, `invalidatePattern`, `invalidateCatalog`; key families `products:school:*`, `product:*`,
`categories:*`, `home:all`, `media:*`, `price:*`; `CACHE_DISABLED=1` bypass.

09 Background jobs & crons

9.1 Job inventory

Endpoint	Schedule (prod)	Module	Auth · lock TTL	Notes
erp-drain	*/1 + 30 s offset	erp-drain.ts	requireCron · 30 s	Push queue; DLQ
erp-poll	*/1 + 30 s offset	erp-poll.ts	requireCron · 25 s	Pull mirror; project status
erp-reconcile	*/10	erp-reconcile.ts	requireCron · 60 s	Re-enqueue missed order.created
erp-webhook-drain	*/1	erp-webhook-dispatch.ts	requireCron · 60 s	Processes stored inbound webhooks (scheduled 2026-09-23)
erp-stranded-alert	5 * * * *	erp-stranded.ts	requireCron · 60 s	E-mails ALERT_EMAIL_T0 (scheduled 2026-09-23)
ccavenue-reconcile	*/5	ccavenue-finalize.ts	requireCron · 30 s	≤50 lookups/run, ≤5 days old
ccavenue-settlement-reconcile	17 * * * *	ccavenue-settlement-reconcile.ts	requireCron · 120 s	SFTP .xlsx
retry-webhooks	*/2	notify/event-bus.ts	requireCron · 30 s	Outbound webhook redelivery
auto-po	0 2 * * *	auto-po.ts	requireCron · 60 s	Low-stock PO drafts
cleanup-carts	0 */6	repos/cart.ts	requireCron · 60 s	Expired carts; 500 on every tick until 2026-09-23 (F-21, fixed)
ground-stock-sync	*/5	ground-stock-sync.ts	requireCron · 240 s	Redis lock; 240 s timeout
sync-items	manual	jobs/erp-item-sync.ts	requireCron · 60 s	ERPNext item feed; host cannot reach ERPNext, run from the admin ERP page (decision, F-06)

Cron file: one file, /etc/cron.d/inventre-erp (template deploy/erp-cron.example), sources .env.deploy and sends Authorization: Bearer \$CRON_TOKEN on every line; no secret is stored in cron files. **Host scripts:** cron-import-mcb.sh 01:30, backup-db.sh 02:15, cron-import-mcb-receipts.sh 02:30 (flock), snapshot-and-rotate.sh 0,6,12,18 h, monitor-production.sh */10, Redis client-count monitor */5. Dev instance crons in inventre-erp-dev target :3020.

9.2 Backup pipeline (scripts/backup/)

Piece	Runs	What it does
wal-stream.sh (systemd inventre-wal-stream)	always	pg_receivewal in a postgres:16-alpine container on the compose network, slot inventre_wal (created once, separately — --create-slot exits), gzip, to /root/backups-archive/wal. pg_hba.conf allows replication inventre 172.16.0.0/12 .
base-backup.sh	00:00 IST	pg_basebackup -Ft -z -X none → base/<stamp>/ ; prunes base > 30 d and WAL > 31 d.
code-bundle.sh	every commit (.git/hooks/post-commit , copy in deploy/git-hooks-post-commit.sh), after each deploy, 00:20	git bundle --all → code/ (keep 14); git push --all/--tags to git@github.com:Sunith0801/Inventre-Website.git with /root/.ssh/inventre_github_deploy (live since 2026-09-23 17:38).
backup-db.sh	00:10 IST	Logical pg_dump -Fc → db_backups/daily (14) + monthly ; size and object-count sanity checks.
sync-to-m365.sh	*/5	First re-encrypts .env.deploy to config/env.deploy.gpg when its hash changed, packs uncommitted edits to code/uncommitted-latest.{tar.gz,patch,txt} when the tree is dirty, then rclone sync of wal, base, code, config, db_backups/{daily,monthly} , snapshots → m365crypt: (crypt over m365:Inventre Backups/prod), then publish-recovery-kit.sh . Remote defined from env only (no config file). Alerts on failure and when no WAL for 30 min while the streamer is down.
verify-restore.sh	Sun 03:30	Newest daily dump → throwaway Postgres → table/row counts → e-mail pass/fail.
publish-recovery-kit.sh	each sync	Writes RESTORE/ (unencrypted, next to prod/): README, bootstrap-new-server.sh , restore-from-m365.sh , recovery-kit.env.gpg (the seven M365 / crypt / snapshot values, AES-256 with RECOVERY_KIT_PASSPHRASE).

Piece	Runs	What it does
<code>bootstrap-new-server.sh</code>	on a NEW server	Asks for the recovery-kit passphrase once, then runs <code>restore-from-m365.sh</code> <code>fetch</code> → <code>code</code> → <code>restore-pitr</code> (or <code>dump</code>) → <code>start</code> and prints the manual tail.
<code>lib.sh</code>	—	Sources <code>.env.deploy</code> ; resolves and caches the SharePoint drive id via Graph; <code>mail_alert</code> through msmtpt.
<code>restore-from-m365.sh</code>	on a NEW server	<code>fetch</code> → <code>code</code> → <code>restore-dump</code> <code>restore-pitr</code> <code>[time]</code> → <code>start</code> : downloads dump, base backup, WAL, code bundle and snapshot from the encrypted mirror, clones the code, decrypts <code>.env.deploy</code> , rebuilds the database (simple or point-in-time) and starts the stack on :3010. Needs only the seven password-manager values. Runbook: <code>docs/runbooks/restore-from-m365.md</code> (≈ 60–90 min end to end).

Restore to a point in time (what `restore-pitr` automates): untar the newest `base/<stamp>/base.tar.gz` into an empty data directory, place the `wal/` segments (gunzip) in a restore directory, set `restore_command = 'cp /restore/%f %p'` and `recovery_target_time`, start Postgres. From the remote: `rcclone copy m365crypt:base/<stamp> ...` with the same `BACKUP_CRYPT_*` values.

9.3 Design rules for a cron endpoint

1. First statement: `const denied = requireCron(req); if (denied) return denied; ( server/cron-auth.ts : Authorization: Bearer CRON_TOKEN , constant-time; 503 when unset, 401 when wrong).` The architecture test fails the build if a cron route skips it or reads a retired `CRON_SECRET` / `CRON_KEY`.
2. Second statement: `const lock = await acquireCronLock("<job>", ttlSeconds); if (!lock) return cronLockedResponse("<job>");` and the body in `try ... finally { await lock.release(); }`. Redis `SET NX PX` with a random token, released by compare-and-delete; TTL = the cron line's curl timeout. If Redis is unreachable the job runs unlocked (one instance today; every job is idempotent or uses `FOR UPDATE SKIP LOCKED`). Verified: two simultaneous `erp-poll` calls, one answered `{skipped: "locked"}`.
3. Bound the work per tick (chunk size, max lookups) so a tick finishes inside the curl timeout (25–240 s).
4. Return a JSON summary (`{processed, failed, skipped}`) that the cron log captures; never 200 on a silent failure.
5. Log to `/var/log/inventre-<job>.log` via the cron line; long-term, expose a heartbeat metric (F-13).

10 Configuration & secrets

10.1 Files

File	Used by	Notes
<code>.env.local</code>	<code>next dev</code> , <code>tsx</code> scripts	Developer machine; dev checkout's copy points at the audit <i>prod</i> ERP, so be careful.
<code>.env.deploy</code>	<code>docker compose --env-file .env.deploy</code> , cron files that source it	All production secrets. Omitting <code>--env-file</code> on a recreate breaks Redis auth. Mode should be 600.
<code>.env.example</code>	template	Still lists <code>RAZORPAY_*</code> (no code uses them).
<code>docker-compose.deploy.yml</code>	<code>compose</code>	No credentials inside since 2026-09-23: <code>\${POSTGRES_PASSWORD}</code> , <code>\${MINIO_ROOT_PASSWORD}</code> , <code>\${REDIS_PASSWORD}</code> are interpolated from <code>.env.deploy</code> ; SMS credentials come from <code>env_file . deploy.sh</code> reads the DB password the same way.

10.2 Variables by concern (names only)

Concern	Variables
Core	<code>DATABASE_URL</code> (via PgBouncer), <code>DATABASE_DIRECT_URL</code> (migrator), <code>POSTGRES_PASSWORD</code> , <code>MINIO_ROOT_PASSWORD</code> , <code>REDIS_URL</code> , <code>REDIS_PASSWORD</code> , <code>JWT_SECRET</code> , <code>AUTH_SECRET</code> (unused by signing), <code>NODE_ENV</code> , <code>NEXT_DIST_DIR</code> , <code>NEXT_PUBLIC_APP_URL</code> , <code>APP_PUBLIC_URL</code> , <code>PUBLIC_BASE_URL</code>
Cookies / cache	<code>COOKIE_INSECURE</code> , <code>ALLOW_INSECURE_COOKIES_IN_PROD</code> , <code>CACHE_DISABLED</code>
Migrations	<code>RUN_MIGRATIONS_ON_START</code> , <code>MIGRATIONS_REQUIRED</code> , <code>MIGRATE_SKIP_HISTORICAL_SEEDS</code>
Cron / service keys	<code>CRON_TOKEN</code> (the only cron secret; <code>CRON_SECRET</code> and <code>CRON_KEY</code> retired 2026-09-23), <code>SYNC_STAGING_KEY</code> , <code>STAGING_DB_URL</code> , <code>SUPPORT_VIEW_SERVICE_TOKEN</code> , <code>SUPPORT_VIEW_JWT_SECRET</code> , <code>SUPPORT_VIEW_ENABLED</code> , <code>SUPPORT_VIEW_BASE_URL</code> , <code>PORTAL_TOKEN_SECRET</code> , <code>GIFT_CARD_SECRET</code>
Alerts	<code>ALERT_EMAIL_TO</code> (stranded-order e-mail and the production monitor; legacy <code>ALERT_EMAIL</code> still read), <code>STRANDED_GRACE_HOURS</code>
Audit ERP	<code>ERP_TARGET</code> , <code>{STAGING_,PROD_}ERP_{INGEST_URL,API_BASE_URL,WEBHOOK_SECRET,POLL_USER,POLL_PASS}</code> , <code>ERP_BUFFER_DELAY_SECONDS</code> , <code>ERP_DRAIN_*</code> , <code>ERP_POLL_*</code> , <code>ERP_STUCK_SENDING_SECONDS</code> , <code>ERP_SYNC_BOOTSTRAP_HOURS</code> , <code>ERP_INBOUND_ENABLED</code> , <code>ERP_INBOUND_ORDERS_POLL_ENABLED</code> , <code>BACKFILL_CUTOVER_ISO</code>
ERPNext	<code>ERPNEXT_BASE</code> , <code>ERPNEXT_TOKEN</code> , <code>ERPNEXT_TIMEOUT_MS</code> , <code>ERP_FEED_BASE</code> , <code>ERP_FEED_KEY</code> , <code>ERP_API_KEY</code> , <code>ERP_API_SECRET</code> , <code>ERP_BASE_URL</code> , <code>ERP_MEDIA_BACKEND</code> , <code>ERP_MEDIA_LOCAL_DIR</code> , <code>ERP_MEDIA_LOCAL_URL</code> , <code>ERP_ITEM_REFRESH_MAX_PER_TICK</code>
Payments	<code>CCAvenue_MERCHANT_ID</code> , <code>CCAvenue_ACCESS_CODE</code> , <code>CCAvenue_WORKING_KEY</code> , <code>CCAvenue_API_BASE</code> , <code>CCAvenue_REDIRECT_URL</code> , <code>CCAvenue_CANCEL_URL</code> , <code>CCAvenue_STATUS_{API_BASE,ACCESS_CODE,WORKING_KEY}</code> , <code>CCAvenue_SETTLEMENT_DIR</code>
Messaging	<code>SMS_API_URL</code> , <code>SMS_API_USERNAME</code> , <code>SMS_API_PASSWORD</code> , <code>SMS_TRANS_API_USERNAME</code> , <code>SMS_TRANS_API_PASSWORD</code> , <code>MSG91_AUTH_KEY</code> , <code>MSG91_OTP_TEMPLATE_ID</code> , <code>MSG91_ORDER_TEMPLATE_ID</code> , <code>DEV_OTP</code> , <code>OTP_BYPASS_CODE</code> , <code>OTP_TEST_BYPASS</code> , <code>SMTP_HOST/PORT/USER/PASS</code> , <code>RESEND_API_KEY</code> , <code>EMAIL_FROM</code>
Backups (Microsoft 365)	<code>M365_TENANT_ID</code> , <code>M365_CLIENT_ID</code> , <code>M365_CLIENT_SECRET</code> (Entra secret <i>Inventre-prod</i>), <code>M365_BACKUP_SITE</code> , <code>BACKUP_CRYPT_PASSWORD</code> , <code>BACKUP_CRYPT_SALT</code> (rclone-obscured; losing them loses the remote copies), <code>RECOVERY_KIT_PASSPHRASE</code> (the one value kept in the password manager; unlocks the kit that holds all the others)
Storage / media	<code>S3_ENDPOINT</code> , <code>S3_REGION</code> , <code>S3_BUCKET</code> , <code>S3_ACCESS_KEY_ID</code> , <code>S3_SECRET_ACCESS_KEY</code> , <code>S3_PUBLIC_URL</code> , <code>NEXT_PUBLIC_MEDIA_CDN_HOST</code> , <code>NEXT_PUBLIC_CF_IMAGE_RESIZING</code>
MCB	<code>MCB_API_BASE</code> , <code>MCB_API_KEY</code> , <code>MCB_TOKEN_ID</code> , <code>MCB_BRANCH_IDS</code> , <code>MCB_ORGANISATION_ID</code> , <code>MCB_ACADEMIC_YEAR_IDS</code> , <code>MCB_FEE_WINDOW_DAYS</code>
GST documents	<code>EINVOICE_{API_BASE,GSTIN,USERNAME,PASSWORD}</code> , <code>EWAYBILL_{API_BASE,GSTIN,USERNAME,PASSWORD}</code> , <code>MERCHANT_{LEGAL_NAME,GSTIN,ADDR1,CITY,PINCODE,STATE_CODE}</code>
Feature flags	<code>EXCHANGE_TESTER_PHONES</code> , <code>EXCHANGE_OPEN_TO_ALL</code> , <code>RETURNS_NUMBER_SINGLE_SOURCE</code> , <code>SEED_ON_EMPTY</code> (never in prod)

10.3 Rotation procedure

- Copy `.env.deploy` to `/root/.secrets-archive/` (0700).
- JWT:** move the current value to `JWT_SECRET_PREVIOUS`, generate a new `JWT_SECRET`; delete the previous after 7 days. `PORTAL_TOKEN_SECRET` and `GIFT_CARD_SECRET` must stay pinned to the secrets their tokens and codes were issued with.
- App-only secrets** (`AUTH_SECRET`, `SUPPORT_VIEW_JWT_SECRET`, `OTP_BYPASS_CODE`, `CRON_TOKEN`): change the value, run `./scripts/deploy.sh` (full mode re-reads `.env.deploy`). The cron file sources the same file, so cron follows automatically.
- Redis / MinIO:** change the value, deploy the app, then stop + remove the old container and `docker-compose ... up -d --no-deps redis`. Never `--force-recreate` with `compose v1` (F-22).
- Postgres:** `scripts/rotate-db-password.sh` (`ALTER ROLE` → `.env.deploy` → recreate pgbouncer + app → fast deploy → verify).

6. **Shared secrets** (ERP webhook/poll, support-view service token, CCAvenue, R2, SMTP, SMS, MCB): agree a window with the other party, change both sides, verify the integration's next tick.

SECRET HANDLING RULES

Never print or commit values. `.env.deploy` is the single place production secrets live (cron files, compose, scripts all read from it); environment dumps in chat or logs leak them. Retired env backups are in `/root/.secrets-archive` (0700). Open item: rotation of DB, Redis, JWT and gateway keys after the 2026 compromise (F-04).

11 Logging, audit trail & error handling

Mechanism	Where	What it captures
Unhandled errors	<code>instrumentation.ts</code> <code>onRequestError</code> → <code>server/observability/record-storefront-event.ts</code>	Every uncaught exception in a route, page or RSC as <code>storefront_events.kind = 'api.5xx'</code> with router kind, route path, stack and best-effort parent id from the cookie. Shown in the support portal "Failures (24 h)".
Explicit API failures	<code>failJson</code> (5 parent routes)	4xx/5xx returned to the parent are also recorded in <code>storefront_events</code> .
Admin activity	<code>server/activity.ts</code> <code>logActivity</code> ; <code>/admin/activity</code> , <code>RecordHistory</code>	Actor, action, entity, before/after JSON for 14 admin route families and the support-view mint. Full audit trail for admin changes.
Auth events	<code>otp_logs</code> , rate-limit keys	Every OTP send with gateway response; login throttling.
Integration	<code>webhook_deliveries</code> , <code>erp_outbound_queue.last_error</code> , <code>erp_outbound_dlq</code> , <code>erp_webhook_events</code> , <code>ground_stock_sync_runs</code> , <code>order_notifications</code> , <code>communications</code>	Per-attempt request/response, dead letters, run summaries, message sends.
Process logs	<code>docker logs inventre-deploy-app</code> ; cron logs <code>/var/log/inventre-*.log</code> ; monitor log <code>db_backups/production-monitor.log</code>	stdout/stderr, cron JSON summaries, security-posture checks with build id.
Metrics	<code>deploy/monitoring</code> Prometheus (node, postgres, redis, pgbouncer exporters, blackbox), Grafana at <code>grafana.inventre.in</code>	Host and datastore health; HTTP probes. Alerts: the monitor e-mails <code>ALERT_EMAIL_TO</code> via the host msmtplib relay on posture lost / recovered / foreign commit; stranded-order e-mail hourly (F-13).
Health	<code>GET /api/health</code>	<code>{db, redis, timestamp}</code> ; 503 when either fails; Docker healthcheck every 30 s. <code>/api/version</code> returns <code>{buildId, gitSha, gitBranch, dirty, builtAt}</code> .

11.1 Error-handling conventions

- Route handlers return JSON errors with the right status; never HTML. Validation 400, auth 401, permission or CSRF 403, not-found 404, conflict 409 (house colour, duplicate request), gateway 502 for partner failures.
- Telemetry must never break the customer: `onRequestError` swallows its own failures.
- Parent-facing network failures are translated by `errorMessageFor` into plain language.
- Guards that hide data return 404, not 403, when the requester is outside the family graph.

12 Build, test & deployment procedure

12.1 Local development

```
npm install
cp .env.example .env.local
npm run db:up                # postgres :55432, pgbouncer :6432, redis :6379, minio :9000
npm run db:migrate:run       # tsx db/migrate.ts
npm run db:seed              # optional demo data
npm run dev                  # next dev --turbo (NEXT_DIST_DIR=.next-dev keeps prod build apart)
npm run verify               # typecheck + vitest
```

12.2 Quality gates

Gate	Command	Where it runs
Types	<code>npm run typecheck</code> (<code>tsconfig.check.json</code> : app, components, lib, server, db, tests, middleware, instrumentation)	CI, deploy preflight
Unit / domain / architecture	<code>npm test</code> (Vitest, 10 files; invariants: section gates, admin route guards, security headers, no DSN in shipping code, no DB import in client code, root hygiene)	CI, deploy preflight
Lint / format	<code>npm run lint</code> (ESLint 9 flat, next/core-web-vitals; correctness rules error, style rules warn), <code>npm run format:check</code> on changed files	CI
Dependencies	<code>npm audit</code> fails on critical	CI
Build	CI job with Postgres service: migrations from empty (<code>MIGRATE_SKIP_HISTORICAL_SEEDS=1</code>) then <code>next build</code>	CI
Storefront smoke + a11y	<code>npm run smoke</code> (<code>scripts/smoke-storefront.mjs</code>): build to <code>.next-verify</code> , start on :3033 against the dev DB with <code>.env.local</code> , log in with the dev OTP, walk shop → PDP → cart → orders → order detail, screenshot each, run axe-core WCAG A/AA. Header comment has the exact commands.	Before every storefront deploy (manual)
E2E / load	<code>scripts/e2e.ts</code> (Playwright, seeded DB, <code>BASE_URL</code>), <code>loadtest/shop.js</code> (k6 to 20k VUs)	Manual

12.3 Testing environment (testing.inventre.in)

```
cd /root/Inventre-dev && git checkout <branch>
NEXT_DIST_DIR=.next npm run build && pm2 restart inventre-dev # served on :3022
```

12.4 Production deploy

```
cd /root/Inventre                # the ONLY deploy source
git status --porcelain           # must be clean (or --allow-dirty with a reason)
docker tag inventre-app:latest inventre-app:rollback-<topic>-${date +%Y%m%d}
./scripts/deploy.sh --fast        # code-only: build → docker cp → restart
./scripts/deploy.sh              # full: build → compose recreate (uses --env-file .env.deploy) → sync → start
./scripts/verify-deployment.sh    # health, headers, build id AND commit (run automatically by deploy.sh)
tail -f db_backups/production-monitor.log
```

`deploy.sh` exports the commit, branch and dirty flag so the bundle can report them; with `--allow-dirty` the live site reports `dirty:true` , which is visible in the monitor log. If compose must be run by hand:

`docker compose -p inventre-deploy -f docker-compose.deploy.yml --env-file .env.deploy up -d --no-deps app` . Never use `deploy/release.sh` (legacy PM2 path with drizzle push).

12.5 Rollback

```
docker tag inventre-app:rollback-<topic>-<date> inventre-app:latest
docker compose -p inventre-deploy -f docker-compose.deploy.yml --env-file .env.deploy up -d --no-deps app
# or, for a --fast deploy: scripts/restore-snapshot.sh (restores /app/.next.prev)
```

Migrations are forward-only; a rollback that needs schema reversal requires a new down-migration written by hand.

12.6 Audit ERP side (for cross-system changes)

Separate repository and host; deploy by rebuilding its Docker image over SSH. Its dev checkout is untracked, older and diverged — never copy dev → prod there. Frontend overlays by the audit team can clobber changes within the hour; verify the served `index-*.js` hash after deploying.

13 Known limitations & design traps

These are the things that have caused production incidents or wasted engineering days. Each has a rule attached. Rectification items are cross-referenced to the HLD register.

Area	Trap	Rule
Deploy	Build context is the working tree; 16 checkouts once shared the container name; two silent reverts on 2026-09-10	Deploy only from <code>/root/Inventre</code> (all others refuse); commit first so <code>/api/version</code> reports <code>dirty:false</code> ; tag rollback; the verify step fails on a wrong commit (F-01 closed)
Deploy	Recreate without <code>--env-file .env.deploy</code> breaks Redis auth; docker-compose v1 crashes (<code>KeyError: 'ContainerConfig'</code>) on <code>--force-recreate</code> of an existing container and leaves the service stopped (Redis outage 2026-09-23)	Always pass <code>--env-file</code> ; stop + rm, then <code>up</code> ; scripts now use compose v2 (F-22 closed)
Process control	A broad <code>kill -f next-server</code> took production, rewards and pm2 apps down together	Stop dev servers by their own PID or pm2 name only
Migrations	Name-based PK guards in 0014 missed prod's <code>erp_*_pkey</code> names and 502'd the site; a fabricated journal date makes drizzle skip later files in the Rewards app	Guard by table/column existence, not constraint name; test against a prod dump restore
Migrations	<code>erp.*</code> mirror tables are ERP-owned; a rebuilt DB has an empty <code>erp</code> schema	Bootstrap empty mirror tables in a migration (F-19)
Sessions	Signing with <code>AUTH_SECRET</code> verifies nowhere; stale <code>inv_admin</code> on a fee account loops <code>/admin</code> → <code>/fees/login</code>	Only <code>JWT_SECRET</code> ; use <code>/admin/fee-desk-exit</code>
Redirects	Route handlers behind nginx see <code>req.url = 0.0.0.0:3000</code>	Return relative <code>Location</code>
Middleware	10 MB body cap on matched routes → 400 on uploads	Exclude multipart routes from the matcher; authenticate inside
SQL in templates	<code>\s</code> in a JS template literal becomes <code>s</code>	Double the backslashes in <code>sql`</code> regexes
Drizzle	Cannot bind or cast a JS array (42846); a JS <code>Date</code> inside a raw <code>sql`</code> fragment throws in postgres.js (F-21); partial unique indexes do not work with <code>ON CONFLICT</code>	Use <code>sql.join</code> ; use typed operators (<code>lt</code> , <code>and</code>) for dates; use anti-joins
Money	Exports and DB values are paise; audit CP lives on the keeper page, not <code>_valuation_rows</code>	Label units; take CP from <code>keeper_cost_price</code>
Grades	ERP labels carry +3; bulk imports store real grades; audit takes <code>students.grade</code> verbatim	<code>students.grade</code> is canonical; never <code>grade_snapshot</code>
ERP sync	Stale seq → 409 swallowed; audit drops sub-items for sealed orders but returns 200; manual audit requests reach us only via webhooks	Verify in the audit DB; bump <code>erp_event_seq</code> for dev e2e
Catalogue	Magic Box <code>bundle_selections</code> were lost in a cart race (63 % NULL) and backfilled from components; free bookkit qty > 1 loophole	All-leaf guard; <code>capToOne</code> ; one Magic Box per student
Family graph	Login showed a sibling's data via shared-guardian bleed; YIPS phone collisions mis-linked orders	Scope every read to the requesting graph; match on enrolment + phone, never name
Rendering	Storefront is client-rendered; server actions break open tabs on deploy	Prefer API routes; <code>VersionWatcher</code> is mounted on shop, admin and fees (F-16 closed)
Storefront code	Never edit <code>components/shop</code> , <code>app/shop</code> , <code>app/api/shop</code> , <code>bundle-engine</code> without seeing it run first	Live-site caution: <code>npm run smoke</code> on a local build against the dev DB (or testing.inventre.in), then deploy with a rollback tag
Jobs	<code>background_jobs</code> + <code>server/jobs</code> run detached with <code>setImmediate</code> ; a restart loses them. Rotating <code>CRON_TOKEN</code> needs a full deploy (env change) and the cron file sources the same file, so the two switch together	Keep jobs idempotent and re-runnable from admin; treat cron endpoints as the real scheduler; after a rotation expect one 401 in the cron log during the container recreate
Backups	The old R2 upload failed silently for a week; <code>pg_receivewal --create-slot</code> exits after creating the slot (looked like a crash loop); a replication connection needs its own <code>pg_hba.conf</code> line	Watch <code>/var/log/inventre/m365-sync.log</code> and the alert mails; the streamer creates the slot in a separate step; never delete slot <code>inventre_wal</code> while the service runs (F-11)

Area	Trap	Rule
Third-party data	MCB <code>Status1</code> is the student's status <i>now</i> ; a blanket Active filter deletes paid history	Leavers drop out of outstanding only, live year only
Docs	Superseded plans (Better Auth, Razorpay, BullMQ, Caddy, Coolify) now live in <code>docs/history/</code>	This LLD and the HLD are the reference (F-14 closed)